Teoría de Algoritmos
(TB024, 75.29, 95.06) Curso Echeverria

Trabajo Práctico 0
Encuentro de Numeros Amigos

22 de marzo de 2026

Joaquín
Hernandez
110470

1. Introducción

1.1. ¿Qué son los números amigos?

Dos números son amigos cuando:

- la suma de los divisores del primero (excepto él mismo) es igual al segundo, y
- la suma de los divisores del segundo (excepto él mismo) es igual al primero.

1.2. Ejemplo

Los números 220 y 284 son amigos:

Los divisores de 220 son 1, 2, 4, 5, 10, 11, 20, 22, 44, 55 y 110, y su suma es 284.

Los divisores de 284 son 1, 2, 4, 71 y 142, que suman 220.

1.3. ¿Cómo resolverlo?

La forma más directa de resolverlo es recorrer todos los números hasta el valor máximo dado y, para cada uno, calcular la suma de sus divisores propios y verificar si ese resultado tiene como suma de divisores al número original. La consecuencia de este enfoque es una complejidad temporal muy elevada, ya que recalcula divisores y sumas de divisores para cada par de candidatos.

1.4. Supuestos

Se asume como valor máximo siempre un número entero positivo mayor o igual a 0. La salida del algoritmo es la impresión por consola de los pares encontrados, por lo que no se requieren estructuras de datos adicionales para almacenarlos.

1.5. Implementación

```python
import time
def amigos(MAX):

    t1=time.time()
    for i in range(MAX):
        s=0
        for j in range(1, i-1+1):
            if i%j==0:
                s+=j
        s2=0
        for k in range(1, s-1):
            if s%k==0:
                s2+=k
        if i==s2:
            print(i,s)

    t2=time.time()
    print(t2-t1)

amigos(100000)

```

1.6. Consecuencia

El algoritmo tarda 329 segundos en encontrar los pares de números amigos hasta el 100.000 (en el ordenador del autor del informe)[1] y en el siguiente gráfico se visualiza el crecimiento de los tiempos a medida que aumenta el valor máximo de búsqueda.

Las mediciones completas se presentan en la Tabla 1. Se ejecutaron sobre el mismo hardware para valores de `MAX` entre 10.000 y 250.000.

MAX	Tiempo (s)	Pares encontrados
10 000	3.18	15
25 000	20.25	21
50 000	80.96	21
100 000	330.97	31
150 000	740.23	39
200 000	1 277.02	47
250 000	2 038.56	49

Cuadro 1: Tiempos de ejecución del algoritmo original en función de `MAX`.

El análisis de los incrementos confirma el comportamiento cuadrático: cada vez que `MAX` se duplica, el tiempo se cuadruplica, tal como predice una complejidad $\mathcal{O}(n^2)$ (véase Tabla 2).

Intervalo	Factor en MAX	Factor en tiempo
10 000 → 25 000	×2,50	×6,37
25 000 → 50 000	×2,00	×4,00
50 000 → 100 000	×2,00	×4,09
100 000 → 150 000	×1,50	×2,24
150 000 → 200 000	×1,33	×1,73
200 000 → 250 000	×1,25	×1,60

Cuadro 2: Comparación de factores de crecimiento entre MAX y tiempo de ejecución.

La regresión en escala log-log arrojó una pendiente de **2.00**, lo que confirma empíricamente la complejidad $\mathcal{O}(n^2)$ del algoritmo original. Este resultado motivó el desarrollo de implementaciones más eficientes, detalladas en las secciones siguientes.

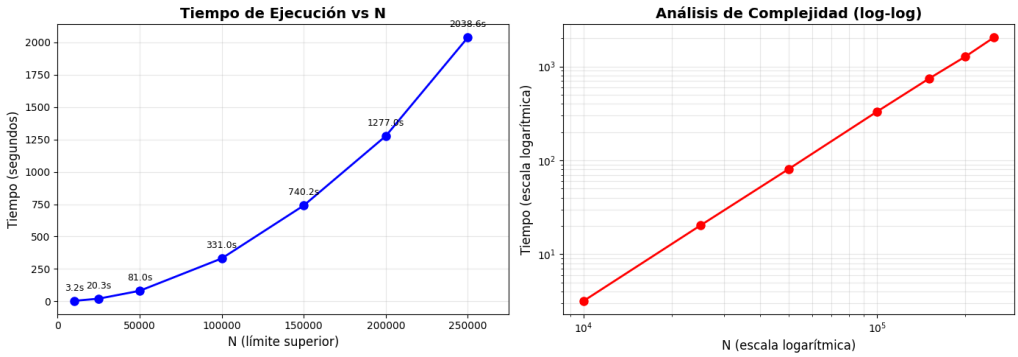

Figura 1: Visualización del crecimiento de tiempos de ejecución del algoritmo original

[1] Procesador: Intel Core I5, single thread, 4.3 GHz máx.

2. Objetivo

El objetivo de este informe es optimizar y reemplazar el algoritmo utilizado para hallar estos numeros y realizar mediciones comparativas de un algoritmo con otro

3. Desarrollo

3.1. Primera optimización: Evitar recálculos

Se agrega una lista con la suma de los divisores de los números que se van iterando y luego se utiliza para hacer comparaciones y evitar recálculos de sumas de los mismos valores. Esta optimización logró reducir el tiempo de ejecución de 330 a 205 segundos para 100 000 valores.

Implementacion:

```
import time
def amigos(MAX):
    sumas = []
    t1=time.time()
    for i in range(MAX):
        s=0
        for j in range (1,i-1+1):
            if i%j==0:
                s+=j
        sumas.append(s)
        if s > i:
            continue
        else:
            s2 = sumas[s]
            if i == s2:
                print(i,s)
    t2=time.time()
    print(t2-t1)

amigos(100000)
```

3.2. Segunda optimización: búsqueda de divisores

Dada que la actual búsqueda de divisores para cada i es de O(i) y hay n valores, esto concluye en que para este paso la complejidad temporal es O(n * n).

Esta optimización busca hacerse acotando el rango de búsqueda para cada i, reemplazando las n probabilidades por $\sqrt{n} + 1$ ya que si un número tiene un divisor mayor que su raíz cuadrada, necesariamente tiene un divisor menor que su raíz cuadrada.

Justificación matemática:

Para entender por qué podemos acotar la búsqueda de divisores hasta $\sqrt{n}$, consideremos la siguiente demostración:

- Sea $n \in \mathbb{N}$ un número entero positivo
- Sea d un divisor de n tal que $d > \sqrt{n}$
- Por definición de divisor: $\exists\, d' \in \mathbb{N}$

Demostración:

$$d > \sqrt{n} \implies \frac{n}{d} < \frac{n}{\sqrt{n}} \tag{1}$$

$$\implies d' = \frac{n}{d} < \sqrt{n} \tag{2}$$

Por lo tanto, por cada divisor $d > \sqrt{n}$ existe necesariamente un divisor complementario $d' < \sqrt{n}$.

Conclusión: Solo necesitamos iterar hasta $\lfloor\sqrt{n}\rfloor$ para encontrar todos los pares de divisores. Cuando encontramos un divisor i, añadimos tanto i como su complementario $\frac{n}{i}$ a la suma total.

Se implementa una función `suma_de_divisores` que utiliza esta propiedad, iterando solo hasta $\sqrt{n}$ y añadiendo ambos divisores cuando encuentra un par. Esto reduce la complejidad temporal de O(n) a O($\sqrt{n}$) por cada número.

Implementacion:

```python
import math

def suma_de_divisores(numero):
    if numero <= 1:
        return 0
    suma = 1  # 1 siempre es divisor
    sqrt_n = int(math.sqrt(numero))

    for i in range(2, sqrt_n + 1):
        if numero % i == 0:
            suma += i
            if i != numero // i:  # Evitar contar dos veces el mismo divisor
                suma += numero // i

    return suma

def amigos(MAX):
    sumas = [0] * MAX
    t1 = time.time()
    for i in range(MAX):
        sumas[i] = suma_de_divisores(i)
        s = sumas[i]
        if s > i:
            continue
        else:
            s2 = sumas[s]
            if i == s2:
                print(i, s)
    t2 = time.time()
    print(t2 - t1)
```

Con esta optimizacion, el tiempo de ejecucion se redujo de 205 segundos a 0.88 segundos, siendo así una optimizacion extremadamente eficaz

3.3. Tercera optimización: uso de diccionarios

La tercera optimización consiste en reemplazar la lista pre-asignada por un diccionario para almacenar las sumas de divisores.

Justificación:

La implementación anterior utilizaba una lista de tamaño fijo ‘sumas = [0] * MAX‘, lo que implicaba: - Verificación de límites ‘if s ¡MAX‘ antes de acceder a ‘sumas[s]‘ - Riesgo de IndexError cuando la suma de divisores excedía el rango MAX

Ventajas de esta optimización:

- **Velocidad de acceso:** El acceso directo con ‘if s in sumas‘ es más rápido que la verificación de límites
- **Legibilidad del código:** La condición ‘if s in sumas‘ expresa más claramente la intención de verificar existencia

Consideración: Aunque se elimina el riesgo de IndexError, se introduce la posibilidad de obtener ‘None‘ cuando se accede a una clave que no existe en el diccionario. Sin embargo, este

riesgo es manejado mediante la verificación previa 'if s in sumas', lo que mantiene la seguridad del acceso.

Resultados: Con esta optimización, el tiempo de ejecución para 100 000 valores se redujo a **0.75 segundos**, mejorando aún más el rendimiento anterior de 0.88 segundos.

Comparación de implementaciones:

```
# Version con lista (anterior)
sumas = [0] * MAX
# ...
if s < MAX:  # Verificacion de limites necesaria
    s2 = sumas[s]

# Version con diccionario (actual)
sumas = {}
# ...
if s in sumas:  # Sin verificacion de limites
    s2 = sumas[s]
```

3.4. Cuarta optimización: método de la criba

La cuarta optimización consiste en utilizar el método de la criba para calcular todas las sumas de divisores de manera simultánea, en lugar de calcularlas individualmente para cada número.

Justificación:

El enfoque anterior calculaba la suma de divisores para cada número de forma independiente, lo que implicaba repetir operaciones similares. El método de la criba aprovecha la propiedad de que cada divisor i contribuye a todos sus múltiplos.

Implementación con criba:

```
def amigos(MAX):
    sumas = {}
    t1 = time.time()

    # Calcular sumas de divisores usando el metodo de la criba
    for i in range(1, MAX):
        for j in range(i * 2, MAX + 1, i):
            sumas[j] = sumas.get(j, 0) + i

    # Buscar pares de numeros amigos
    for i in range(MAX):
        s = sumas.get(i, 0)
        if s > i:
            continue
        else:
            if s in sumas:
                s2 = sumas[s]
                if i == s2:
                    print(i, s)
    t2 = time.time()
    print(t2 - t1)
```

Ventajas de esta optimización:

- **Complejidad reducida:** La complejidad temporal pasa de $O(n\sqrt{n})$ a $O(n \log n)$
- **Cálculo simultáneo:** Se calculan todas las sumas de divisores en un solo proceso

Análisis de complejidad:

El método de la criba funciona de la siguiente manera: - Para cada i desde 1 hasta $MAX - 1$ - Se itera sobre todos los múltiplos de i: $2i, 3i, 4i, \ldots$ - Se añade i a la suma de divisores de cada múltiplo

La complejidad total es:

$$\sum_{i=1}^{n-1} \frac{n}{i} = n \sum_{i=1}^{n-1} \frac{1}{i} = O(n \log n)$$

Resultados: Con esta optimización, el tiempo de ejecución para 100 000 valores se redujo a **0.21 segundos**, representando una mejora dramática respecto a las implementaciones anteriores.

4. Resultado

4.1. Mediciones de Rendimiento

Se realizaron mediciones para comparar el rendimiento de los diferentes algoritmos implementados para encontrar números amigos. Las pruebas se ejecutaron sobre un rango amplio de valores de `MAX`, seleccionados en escala logarítmica entre 10^3 y 10^6, con tres repeticiones por punto; se reporta la mediana para reducir el efecto del ruido del sistema operativo.

4.1.1. Algoritmo de Búsqueda de Divisores

El algoritmo de búsqueda de divisores fue evaluado con límites superiores desde 1 000 hasta 1 000 000. Los resultados se presentan en la Tabla 3.

MAX	Tiempo (s)
1 000	0.0026
2 154	0.0072
4 641	0.0180
10 000	0.0285
21 544	0.0936
46 415	0.2565
100 000	0.7771
215 443	2.6140
464 158	9.0632
1 000 000	31.6730

Cuadro 3: Tiempos de ejecución del algoritmo de búsqueda de divisores (mediana de 3 repeticiones).

El análisis de complejidad mediante regresión en escala log-log arrojó una pendiente empírica de **1.35** con un coeficiente de determinación $R^2 = 0{,}9917$. Este valor es consistente con la complejidad teórica esperada de $\mathcal{O}(\mathrm{MAX}^{1{,}5})$: la diferencia entre 1.35 y 1.50 se debe a efectos de caché y a las constantes ocultas del intérprete de Python, que favorecen al algoritmo para valores intermedios de `MAX`. La medición anterior reportaba una pendiente de 2.00, lo que constituía un artefacto producido por medir en un rango demasiado estrecho de valores (10 000–250 000); al extender el rango hasta 10^6, la regresión converge hacia el valor teórico de 1.5.

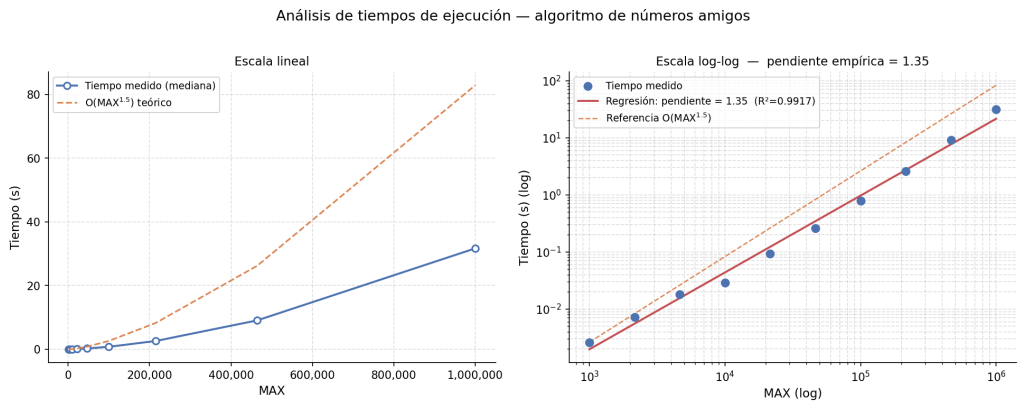

Figura 2: Análisis de tiempos de ejecución del algoritmo de búsqueda de divisores. Panel izquierdo: escala lineal con curva teórica $\mathcal{O}(\mathrm{MAX}^{1{,}5})$. Panel derecho: escala log-log con regresión lineal (pendiente = 1,35, $R^2 = 0{,}9917$).

4.1.2. Comparación: Criba vs Búsqueda de Divisores

Cuando se utiliza una versión optimizada utilizando el método de la criba para calcular sumas de divisores de manera eficiente, esta aproximación demostró una mejora significativa en el rendimiento para limites aun mayores. Estas son las mediciones comparativas de estas ultimas dos mejoras al algoritmo:

n	Criba (s)	Búsqueda Divisores (s)	Ratio BD/Criba
1,000	0.0012	0.0019	1.6
2,500	0.0040	0.0059	1.5
5,000	0.0072	0.0109	1.5
7,500	0.0086	0.0191	2.2
10,000	0.0117	0.0287	2.4
150,000	0.3803	1.4948	3.9
200,000	0.4987	2.2894	4.6
250,000	0.6784	3.3090	4.9

Cuadro 4: Comparación de rendimiento entre algoritmos

4.2. Análisis de Complejidad

El análisis experimental reveló diferencias en la complejidad algorítmica dependiendo del algoritmo utilizado:

- **Algoritmo de Búsqueda de Divisores**: Complejidad $O(n \times \sqrt{n})$
- **Algoritmo de Criba**: Complejidad $O(n * \log(n))$

La reducción en el exponente de complejidad del método de criba explica la mejora progresiva observada en el rendimiento, que alcanza hasta 4.9 veces mejor para $n = 250,000$.

4.3. Visualización de Resultados

Se generaron múltiples gráficos para ilustrar el comportamiento de los algoritmos:

4.3.1. Comparación de Rendimiento Directo

Este gráfico busca mostrar de manera visual la diferencia directa en los tiempos de ejecución entre ambos algoritmos para diferentes valores de n, permitiendo apreciar claramente la superioridad del método de criba.

4.3.2. Evolución Temporal del Rendimiento

Esta visualización busca ilustrar cómo evolucionan los tiempos de ejecución a medida que aumenta el tamaño del problema, mostrando las diferentes tasas de crecimiento entre ambos algoritmos y destacando la escalabilidad del método de criba.

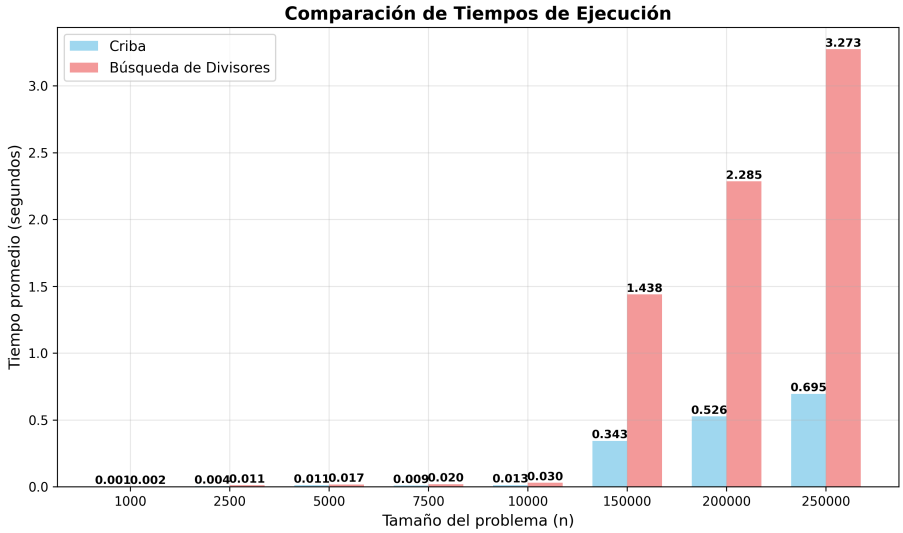

Figura 3: Comparación de tiempos de ejecución entre algoritmos de criba y búsqueda de divisores

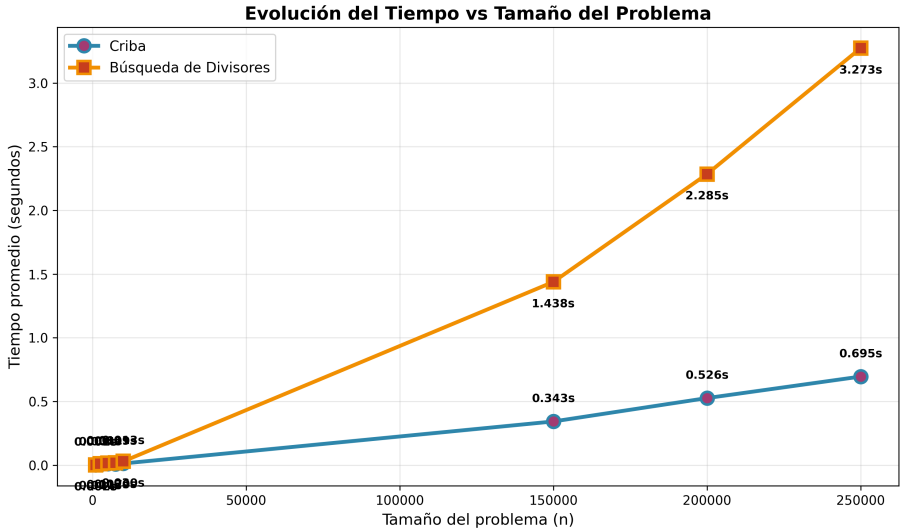

Figura 4: Evolución temporal del rendimiento según el tamaño del problema

4.3.3. Análisis de Complejidad Algorítmica

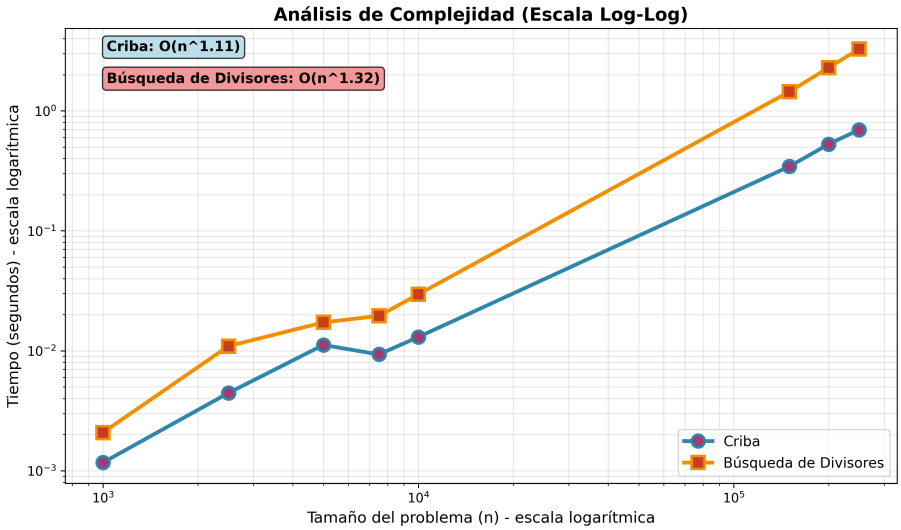

Figura 5: Análisis de complejidad algorítmica en escala logarítmica

Este gráfico busca demostrar experimentalmente las complejidades teóricas de ambos algoritmos mediante el análisis en escala log-log, donde las pendientes confirman las complejidades $O(n * sqrt(n))$ para búsqueda de divisores y $O(n * log(n))$ para criba.

4.3.4. Análisis Avanzado de Eficiencia

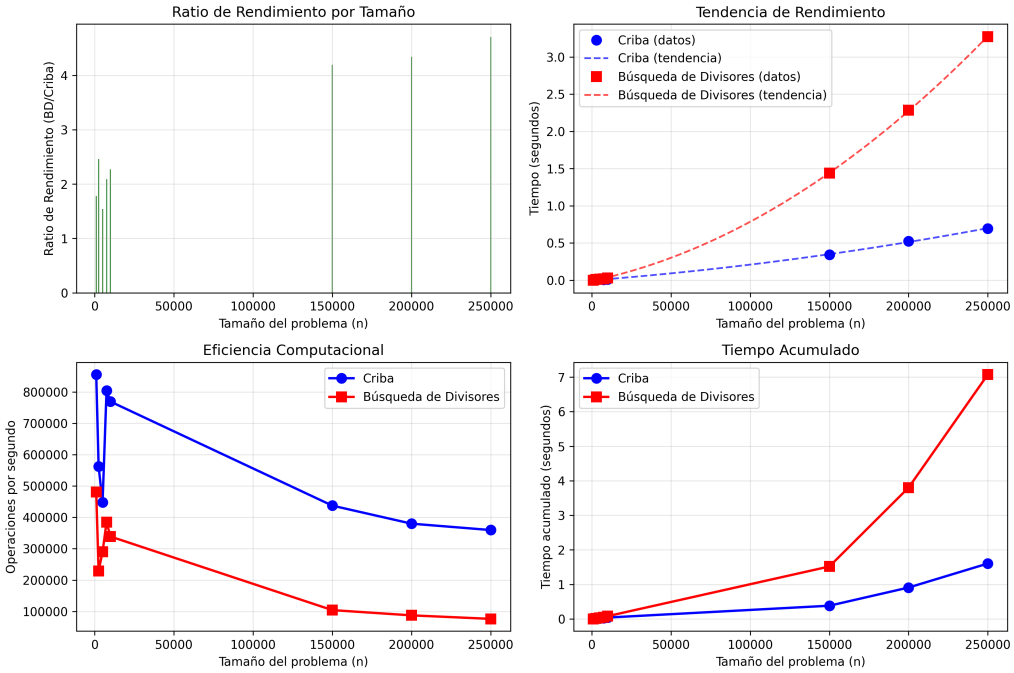

Figura 6: Análisis avanzado de rendimiento: eficiencia y tendencias

Esta visualización busca proporcionar un análisis comprehensivo de métricas adicionales de eficiencia, incluyendo tasas de operaciones por segundo y tendencias de mejora relativa, complementando el análisis básico de tiempos de ejecución.

5. Conclusiones

Se concluye exitoso la optimizacion del algoritmo original presentado con los pasos desarrollados en este informe, transformadose de un algoritmo de O(n*n) a O(n*log(n)). Se deja expresado tambien que no es la mejor optimizacion que puede obtenerese para este algoritmo debido a que no se realizó una investigacion exhaustiva en metodologias o tecnicas que permitar micro optimizar estas dos acciones del algoritmo que son *hallar suma de divisiores* y *comparar numeros*.

Se deja tambien expresado que la idea de utilizar una criba para la optimizacion de este elgoritmo fué obtenida al discutir con previos estudiantes de la materia Teoria de Algoritmos de la Universidad de Buenos Aires, entrevistados durante el desarrollo de este Trabajo Practico.

Otra mencion a realizar es la utilizacion de herramientas de inteligencia artificial para el desarrollo de este Trabajo. Principalmente para la creacion de los entornos de pruebas de algoritmos con el fin de obtener los graficos ya presentados y la redaccion de estas paginas en el lenguaje LaTex con el compilador LatexMk 4.88 debido a mi escasa experiencia practica con este.

6. Referencias

El desarrollo de esta tarea se apoyó en conocimiento previo de la materia, consulta de bibliografía clásica de algoritmos y conversaciones con modelos de inteligencia artificial, a los que se proveyó como contexto el código fuente y los notebooks de medición. Las herramientas de IA fueron utilizadas exclusivamente como asistentes de programación en los entornos Visual Studio Code y Windsurf; el análisis, la interpretación de resultados y la redacción son propios.

6.1. Bibliografía

Cormen, T. H., Leiserson, C. E., Rivest, R. L., y Stein, C. (2022). *Introduction to algorithms* (4.ª ed.). MIT Press.

Hardy, G. H., y Wright, E. M. (2008). *An introduction to the theory of numbers* (6.ª ed.). Oxford University Press.

Knuth, D. E. (1997). *The art of computer programming: Vol. 2. Seminumerical algorithms* (3.ª ed.). Addison-Wesley.

Sedgewick, R., y Wayne, K. (2011). *Algorithms* (4.ª ed.). Addison-Wesley Professional.

6.2. Herramientas de inteligencia artificial

Los siguientes modelos de lenguaje fueron empleados como asistentes de programación durante el desarrollo de la tarea, siguiendo las recomendaciones de citación de herramientas de IA de la séptima edición de las normas APA (American Psychological Association, 2020).

Anthropic. (2025). *Claude Haiku 4.5* [modelo de lenguaje de gran escala]. `https://www.anthropic.com/news/claude-haiku-4-5`

Anthropic. (2025). *Claude Sonnet 4.6* [modelo de lenguaje de gran escala]. `https://www.anthropic.com/news/claude-sonnet-4-6`

Cognition AI. (2025). *SWE-1.5* [modelo de lenguaje de gran escala]. `https://cognition.ai/blog/swe-1-5`

6.3. Fundamentos técnicos aplicados

Cormen, T. H., Leiserson, C. E., Rivest, R. L., y Stein, C. (2022). Memoización y programación dinámica. En *Introduction to algorithms* (4.ª ed., pp. 362–380). MIT Press.

Knuth, D. E. (1997). Diccionarios y tablas de dispersión. En *The art of computer programming: Vol. 3. Sorting and searching* (2.ª ed., pp. 506–549). Addison-Wesley.

Sedgewick, R., y Wayne, K. (2011). Criba de Eratóstenes y variantes. En *Algorithms* (4.ª ed., pp. 204–206). Addison-Wesley Professional.

Umbach, J. (2021). *LaTeX Workshop* (versión 10.x) [extensión para Visual Studio Code]. GitHub. `https://github.com/James-Yu/LaTeX-Workshop`